

UNIVERSIDAD DE BURGOS

REGIONAL UNIVERSITY NETWORK
EUROPEAN UNIVERSITY

GRADO EN INGENIERÍA INFORMÁTICA

Universidad de Burgos

ASIGNATURA

Programación Modular y Orientada a Objetos

TRABAJO / PRÁCTICA

Práctica I: Refactorización Orientada a Objetos de una Planta Industrial

Estudiante	Alejandro Venancio Calviño
DNI / NIE	54156572J
Correo electrónico	avc1024@alu.ubu.es
Grupo	Grupo 1
Profesor/a	Jaime
Curso académico	2026–2027
Fecha de entrega	4 de noviembre de 2026

Índice

1. Introducción **3**

2. Objetivos **3**

2.1. Objetivo general 3

2.2. Objetivos específicos 3

3. Análisis del problema **4**

3.1. Requisitos funcionales 4

3.2. Restricciones y consideraciones 4

4. Diseño orientado a objetos **5**

4.1. Identificación de clases 5

4.2. Diagrama de clases UML 5

4.3. Relaciones entre clases 6

5. Principios de programación orientada a objetos aplicados **6**

5.1. Encapsulación 6

5.2. Abstracción 6

5.3. Modularidad y Separación de Responsabilidades 6

6. Implementación **6**

6.1. Estructura del proyecto 6

6.2. Fragmentos de código relevantes 7

6.2.1. CintaTransportadora: Definición de Direcciones 7

6.2.2. CamaraInspeccion: Detección Reactiva y Variable Booleana . . 7

6.2.3. Horno: Modificación Dinámica del Color en MuJoCo 8

7. Manejo de errores y validaciones **8**

8. Pruebas y validación **9**

9. Resultados **9**

10.Discusión **10**

11.Conclusiones **10**

12.Contribución individual **10**

13.Declaración sobre el uso de herramientas de inteligencia artificial **11**

14.Referencias **11**

A. Código fuente y repositorio **12**

A.1. Instrucciones de ejecución 12

B. Material adicional: Tabla de Controles **12**

1. Introducción

En el ámbito de la automatización industrial moderna, la simulación virtual permite validar arquitecturas de control y algoritmos de visión artificial antes de su despliegue físico. Esta práctica aborda el rediseño y refactorización de una simulación completa de una planta industrial desarrollada en Python. La planta integra el motor de física tridimensional **MuJoCo**, procesamiento de visión computacional mediante **OpenCV** y cálculos matriciales con **NumPy**.

El código original proporcionado cumplía con los requisitos funcionales básicos pero estaba implementado bajo un paradigma puramente procedural. Las decisiones de transporte, el estado de las cintas, las lecturas de cámaras, el control térmico del horno y el bucle principal de simulación estaban acoplados mediante variables globales (`BANDERA_CINTA_1`, `BANDERA_CINTA_2`, etc.) y funciones monopieza dentro de un bucle maestro `run()`.

El propósito central de esta práctica es transformar dicha implementación procedural en una arquitectura modular limpia orientada a objetos (POO). Se busca que cada componente físico y lógico de la planta (cintas, cámaras de inspección, piezas de producción, horno y coordinador) quede modelado como una clase con atributos de estado encapsulados y responsabilidades bien definidas.

2. Objetivos

2.1. Objetivo general

Transformar la simulación procedural de la planta industrial en un sistema orientado a objetos extensible, mantenible y modular, eliminando variables globales y preparando la arquitectura de software para las tareas avanzadas de la Práctica II.

2.2. Objetivos específicos

1. Diseñar e implementar las clases principales: `CintaTransportadora`, `CamaraInspeccion`, `ObjetoIndustrial`, `Horno` y `PlantaIndustrial`.
2. Encapsular el control de marcha/paro y la cinemática de transporte dentro de cada objeto de tipo `CintaTransportadora` (gestionando correctamente las cintas 1 a 5).
3. Integrar en `CamaraInspeccion` el procesamiento de visión mediante OpenCV, manteniendo la variable booleana `objeto_detectado` para detener automáticamente la cinta correspondiente.

4. Encapsular en la clase `Horno` el control de temperatura y actualizar dinámicamente sus propiedades de color `geom_rgba` en la ventana de MuJoCo.
5. Mantener operativa la cinemática inversa (IK) y el control de la estación robotizada Franka Emika Panda.
6. Separar el punto de entrada (`main.py`) de los módulos de la aplicación dentro del directorio `src/`.

3. Análisis del problema

3.1. Requisitos funcionales

1. **Gestión de Cintas Transportadoras:** El sistema debe gestionar 5 cintas independientes: 4 formando un circuito cerrado rectangular y una 5ª cinta que deriva piezas hacia el horno térmico.
2. **Adquisición y Procesamiento de Imagen:** Cada cámara debe capturar los cuadros renderizados en MuJoCo, clasificar contornos (círculos/cilindros, cuadrados/cajas) y activar la variable booleana `objeto_detectado`.
3. **Control Reactivo de Parada:** Al activarse `objeto_detectado == True`, la cinta sobre la que se encuentra la pieza debe detenerse automáticamente.
4. **Simulación Térmica del Horno:** El horno debe registrar piezas ingresadas, variar su temperatura según la ocupación y modificar su representación gráfica (`geom_rgba`) de azul ($< 50\,°C$) a naranja ($50 - 100\,°C$) o rojo ($\geq 100\,°C$).
5. **Interfaz Interactiva:** Visualización continua del mosaico de cámaras con OpenCV y control manual por teclado (teclas 1..5, P, R, + y -).

3.2. Restricciones y consideraciones

- **Estructura de archivos:** `main.py` debe residir en la raíz y la lógica en la carpeta `src/`.
- **Encapsulación estricta:** Prohibido el uso de banderas globales de control de marcha de cintas.
- **No reimplementación de la IK:** La cinemática inversa del robot Franka Panda provista debe ser utilizada como componente sin modificar su algoritmo interno.

4. Diseño orientado a objetos

4.1. Identificación de clases

Cuadro 1: Clases principales de la planta industrial refactorizada.

Clase	Responsabilidad	Elementos principales
`CintaTransportadora`	Gestionar marcha/paro, velocidad y cálculo de vectores de fuerza de transporte.	`numero`, `velocidad`, `activa`, `DIRECCIONES`, `calcular_fuerza()`.
`CamaraInspeccion`	Adquisición de imagen desde MuJoCo, clasificación de forma con OpenCV y parada reactiva.	`cinta`, `objeto_detectado`, `forma_detectada`, `actualizar()`.
`ObjetoIndustrial`	Tracking de posición de piezas en el espacio y asignación de segmento de cinta.	`indice`, `body_id`, `segmento`, `actualizar_segmento()`.
`Horno`	Registro de piezas térmicas, modelo dinámico de temperatura y actualización de color RGBA.	`temperatura`, `numero_piezas`, `actualizar_color()`.
`PlantaIndustrial`	Coordinador global del bucle de física (500 Hz), renderizado, robot y teclado.	`cintas`, `camaras`, `objetos`, `horno`, `ejecutar()`.

4.2. Diagrama de clases UML

La arquitectura se articula en torno a la clase coordinadora `PlantaIndustrial`, que compone los distintos subsistemas de la planta:

ESTRUCTURA DE ARQUITECTURA POO

`PlantaIndustrial` $\longrightarrow$ Composición (1..$*$) de `CintaTransportadora`
`PlantaIndustrial` $\longrightarrow$ Composición (1..$*$) de `CamaraInspeccion`
`PlantaIndustrial` $\longrightarrow$ Composición (1) de `Horno`
`CamaraInspeccion` $\longrightarrow$ Asociación de control con `CintaTransportadora`

Figura 1: Relaciones de composición y asociación en la planta industrial.

4.3. Relaciones entre clases

- **Composición:** `PlantaIndustrial` contiene y gestiona el ciclo de vida de 5 instancias de `CintaTransportadora`, 5 de `CamaraInspeccion`, 5 de `ObjetoIndustrial` y 1 de `Horno`.
- **Asociación de control:** Cada `CamaraInspeccion` posee una referencia directa a la `CintaTransportadora` sobre la que realiza la inspección para poder invocar `detener()` cuando detecta una pieza.

5. Principios de programación orientada a objetos aplicados

5.1. Encapsulación

Se ha eliminado todo estado global. Atributos como la velocidad, el estado de actividad (`self.activa`) o la temperatura del horno son privados al estado de cada objeto y solo se modifican mediante métodos como `arrancar()`, `detener()`, `conmutar()` o `aumentar_temperatura()`.

5.2. Abstracción

Las operaciones complejas de simulación física y renderizado quedan ocultas. Por ejemplo, `PlantaIndustrial` solo necesita llamar a `cinta.calcular_fuerza(pos, vel)` sin preocuparse por la matemática interna de empuje o centrado de carril.

5.3. Modularidad y Separación de Responsabilidades

La visión artificial queda contenida exclusivamente dentro de `CamaraInspeccion`, la física de movimiento en `CintaTransportadora` y la respuesta gráfica del proceso térmico en `Horno`.

6. Implementación

6.1. Estructura del proyecto

El proyecto mantiene la separación requerida en el enunciado:

- `main.py`: Punto de entrada que instancia y ejecuta `PlantaIndustrial`.
- `requirements.txt`: Dependencias del entorno (`mujoco`, `opencv-python`, `numpy`).
- `src/cinta_transportadora.py`: Clase `CintaTransportadora` (cintas 1 a 5).
- `src/camara_inspeccion.py`: Clase `CamaraInspeccion` y visión OpenCV.
- `src/horno.py`: Clase `Horno` y control de color RGBA.
- `src/objeto_industrial.py`: Tracking de piezas.
- `src/planta.py`: Clase coordinadora `PlantaIndustrial`.
- `src/robot.py`: Módulo de integración y cinemática del robot Panda.

6.2. Fragmentos de código relevantes

6.2.1. CintaTransportadora: Definición de Direcciones

Se corrigió la enumeración interna para soportar las 5 cintas (claves del 1 al 5 en lugar de 0 a 4):

```
class CintaTransportadora:
    DIRECCIONES = {
        1: (+1,  0),   # inferior  +X
        2: ( 0, +1),   # derecha   +Y
        3: (-1,  0),   # superior  -X
        4: ( 0, -1),   # izquierda -Y
        5: (-1,  0),   # rama horno -X
    }
```

6.2.2. CamaraInspeccion: Detección Reactiva y Variable Booleana

```
def actualizar(self, renderer, data) -> np.ndarray:
    imagen = self.capturar(renderer, data)
    self.forma_detectada = self.detectar_forma(imagen)
    self.objeto_detectado = (self.forma_detectada is not None)

    # Requisito 4.4: Detener cinta automáticamente al detectar pieza
    if self.objeto_detectado and self.cinta.esta_activa():
        self.cinta.detener()

    return self._superponer_info(imagen)
```

6.2.3. Horno: Modificación Dinámica del Color en MuJoCo

```
def actualizar_color(self, model) -> None:
    if self.geom_id < 0:
        return
    if self.temperatura < 50.0:
        color = [0.1, 0.3, 1.0, 1.0]   # Azul (frío)
    elif self.temperatura < 100.0:
        color = [1.0, 0.6, 0.1, 1.0]   # Naranja (nominal)
    else:
        color = [1.0, 0.1, 0.1, 1.0]   # Rojo (caliente)
    model.geom_rgba[self.geom_id] = color
```

7. Manejo de errores y validaciones

1. **Validación de identificadores de cinta:** El constructor de `CintaTransportadora` valida que el índice pertenezca explícitamente a las claves permitidas (`1..5`), lanzando un `ValueError` controlado en caso contrario.

2. **Verificación de Geometrías y Actuadores en MuJoCo:** Se comprueba que los IDs retornados por `mj_name2id` sean mayores o iguales a cero antes de intentar modificar memoria física (`geom_rgba` o `data.ctrl`).

3. **Control de imágenes vacías en OpenCV:** Los métodos de procesamiento de imagen validan la forma del cuadro antes de aplicar transformaciones de escala de grises o extracción de contornos.

8. Pruebas y validación

Cuadro 2: Plan de pruebas de la planta refactorizada.

Prueba	Escenario	Resultado esperado	Estado
P01	Ejecución de `python main.py`	Inicialización de MuJoCo y apertura del mosaico OpenCV	Correcto
P02	Pulsación de teclas 1..5	Conmutación individual del estado ON/OFF de la cinta correspondiente	Correcto
P03	Avance de pieza bajo cámara	`objeto_detectado = True` y parada de la cinta	Correcto
P04	Incremento de temperatura con '+'	Cambio visual progresivo del horno de azul a rojo	Correcto
P05	Transferencia de pieza por Robot Panda	Ejecución fluida de cinemática inversa y depósito en Cinta 5	Correcto
P06	Pulsación de tecla R (Reset)	Reinicio de posiciones de objetos, estado del robot y horno	Correcto

9. Resultados

La aplicación refactorizada se ejecuta a una frecuencia de simulación física constante de 500 Hz (`timestep = 0.002s`). La ventana del mosaico de OpenCV muestra en tiempo real las 5 vistas de inspección con sus respectivas etiquetas de estado y detección, junto con la vista panorámica global.

SIMULACIÓN EN EJECUCIÓN

Entorno 3D MuJoCo + Mosaico de Cámaras OpenCV en tiempo real
PD: No pude recogerlo por algún fallo que incluso en el codigo original ocurre y antes no ocurria

Figura 2: Simulación interactiva de la planta industrial y panel de visión.

10. Discusión

La refactorización ha aportado notables ventajas estructurales frente al código procedural original:

- **Mantenibilidad:** Al eliminar variables globales, cualquier modificación en la cinemática de una cinta se realiza exclusivamente en la clase `CintaTransportadora`.
- **Extensibilidad para la Práctica II:** La inclusión de nuevos algoritmos de visión o reglas de clasificación solo requerirá extender `CamaraInspeccion` o añadir subsistemas coordinados por `PlantaIndustrial`.
- **Desacoplamiento:** El motor físico MuJoCo y el renderizado OpenCV se comunican a través de métodos limpios en lugar de manipular variables en el bucle principal.

11. Conclusiones

Se ha completado con éxito la refactorización orientada a objetos de la planta industrial:

1. Se han diseñado e integrado todas las clases solicitadas (`CintaTransportadora`, `CamaraInspeccion`, `ObjetoIndustrial`, `Horno` y `PlantaIndustrial`).
2. Se eliminó por completo el uso de banderas globales para la gestión de cintas.
3. Se implementó la detección de formas con OpenCV y la variable booleana `objeto_detectado` para la parada reactiva automática.
4. Se implementó el control de temperatura y la actualización de color del horno en MuJoCo.
5. El proyecto se encuentra estructurado y listo para abordar los requisitos de la Práctica II.

12. Contribución individual

Cuadro 3: Contribución del estudiante.

Estudiante	**Contribución**
Alejandro Venancio Calviño	Desarrollo completo de la refactorización POO, implementación de clases en `src/`, integración de OpenCV/MuJoCo, realización de pruebas y redacción de la memoria.

13. Declaración sobre el uso de herramientas de inteligencia artificial

Se utilizó una herramienta de inteligencia artificial generativa como apoyo para la revisión de la redacción, la estructuración del informe en formato LaTeX y la resolución de dudas puntuales de formato. El autor del trabajo revisó, comprendió y validó la totalidad del código y los contenidos presentados en esta memoria.

14. Referencias

Referencias

[1] E. Gamma, R. Helm, R. Johnson y J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.

[2] R. C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*, Prentice Hall, 2008.

[3] E. Todorov, T. Erez y Y. Tassa, *MuJoCo: A physics engine for model-based control*, IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2012.

[4] G. Bradski, *The OpenCV Library*, Dr. Dobb's Journal of Software Tools, 2000.

A. Código fuente y repositorio

El código fuente completo del proyecto se encuentra alojado y disponible en el siguiente repositorio público de GitHub:

https://github.com/alejandrovenanciocalvino-cmd/Practica_1_Alejandro_Venancio_POO

A.1. Instrucciones de ejecución

1. Clonar el repositorio:

```
git clone https://github.com/alejandrovenanciocalvino-cmd/Practica_1_Alejand
cd Practica_1_Alejandro_Venancio_POO
```

2. Instalar dependencias:

```
pip install -r requirements.txt
```

3. Ejecutar desde la raíz del proyecto:

```
python main.py
```

B. Material adicional: Tabla de Controles

Cuadro 4: Comandos por teclado interactivos durante la simulación.

Tecla	Función / Acción
`1..5`	Conmutar marcha/paro de la cinta transportadora correspondiente (C_1 a C_5).
`P`	Pausar / reanudar la simulación global.
`R`	Reiniciar simulación (posiciones de objetos, robot y horno).
`+`	Aumentar la temperatura del horno (+10 °C).
`-`	Disminuir la temperatura del horno (-10 °C).
`S`	Guardar captura de pantalla del mosaico de cámaras en `capturas/`.
`Q / Esc`	Salir de la aplicación ordenadamente.